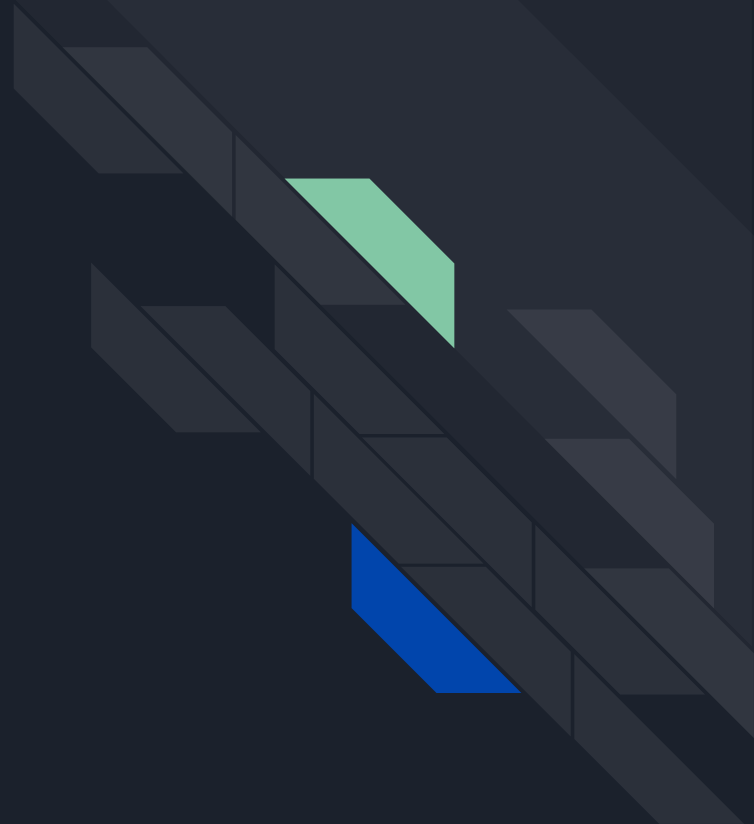


A decorative graphic on the left side of the slide consisting of two overlapping parallelograms. The front one is blue and the back one is a light green. They are positioned diagonally, with the blue one partially covering the green one.

Final Project Design

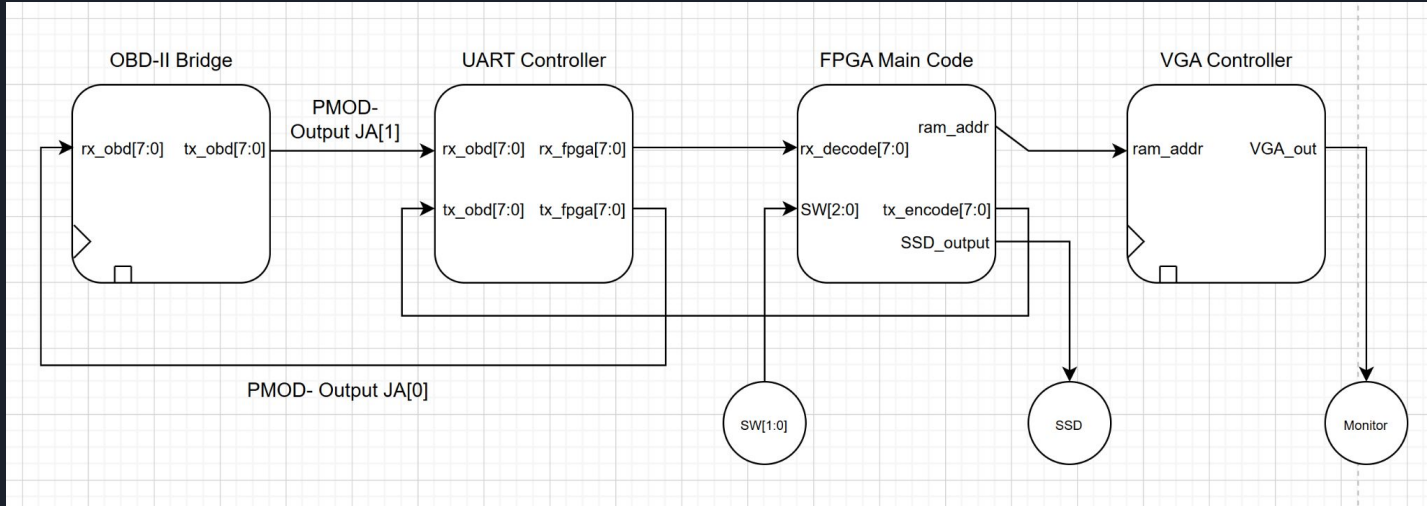
By Abhi Boggavarapu, Trent Jones

Design Summary



Design Summary

Block Diagram: Overall Design





Design Summary

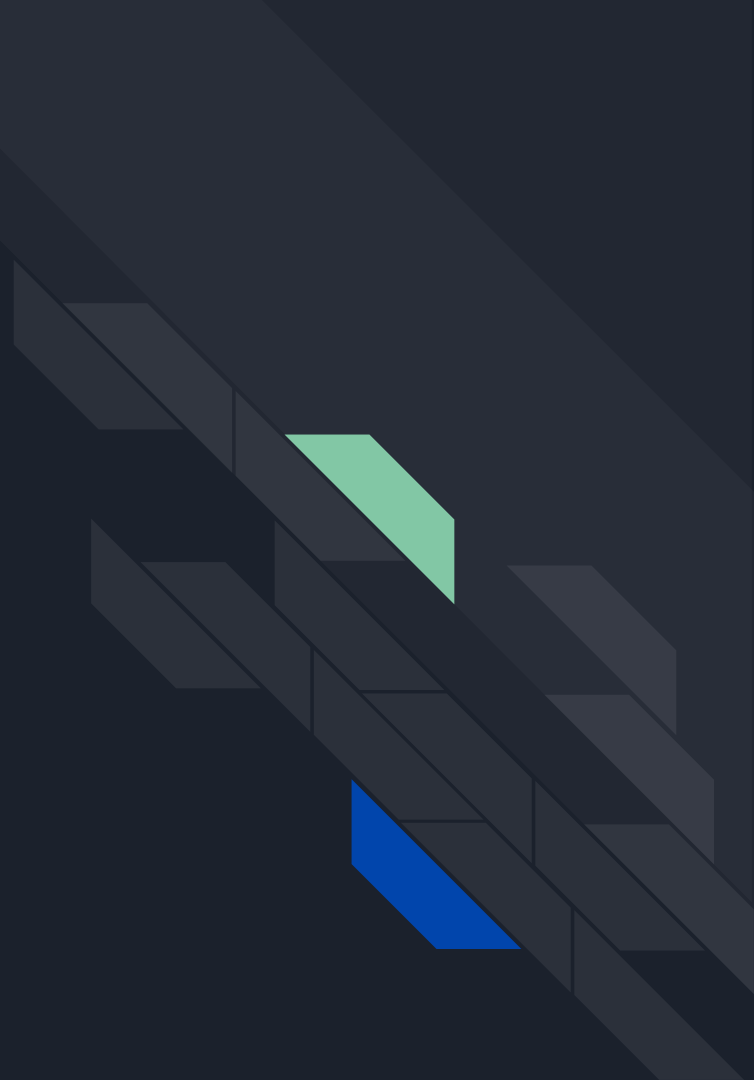
Key Decisions:

- Storing every Nth sample (reduce memory needed for storage)
- Reading-writing onboard DDR2 SDRAM (no external storage needed)
- Manually set baud rate through UART

Problem to be solved:

Many consumers and novice race teams do not have an affordable, quick way to gauge under-the-hood performance of their daily driver or modified hot rod. Our product aims to provide a quick and simple solution to display key statistics of engine performance from coolant temperature to RPM count.

So... What's Changed?



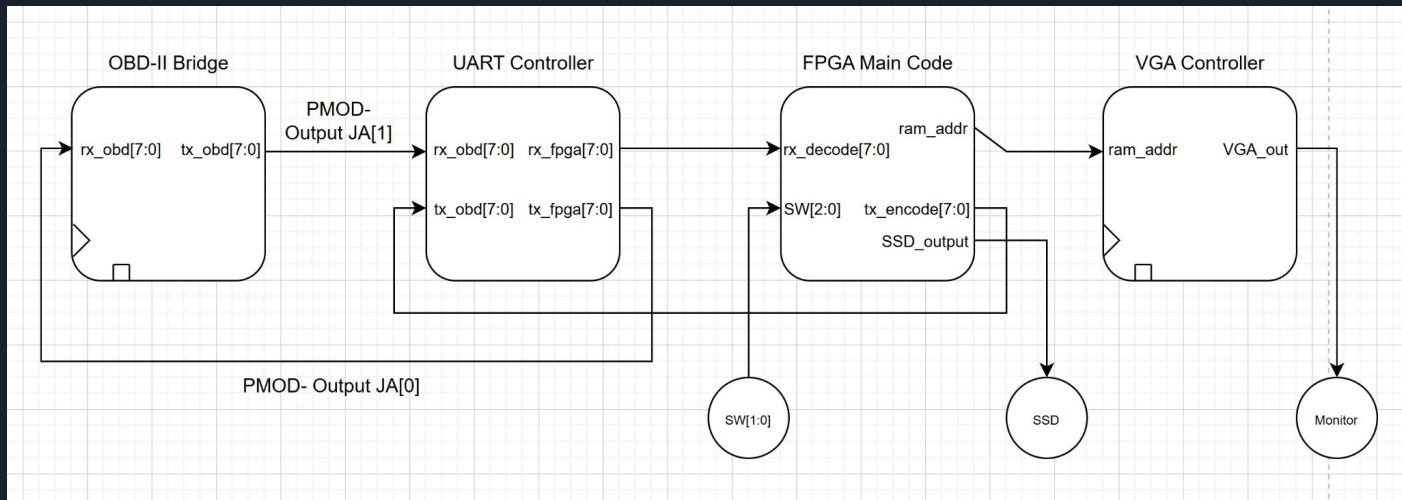


So... What's Changed?

- Taking every Nth sample can reduce the need for expanded storage and allow the use of on board RAM
- The design and protocol has been refined for reading and writing to DDR2 SDRAM, providing a clear basis for accessing and transmitting to VGA output
- The team has developed a timeline for work to be completed by both members in order to meet and exceed previously established deadlines

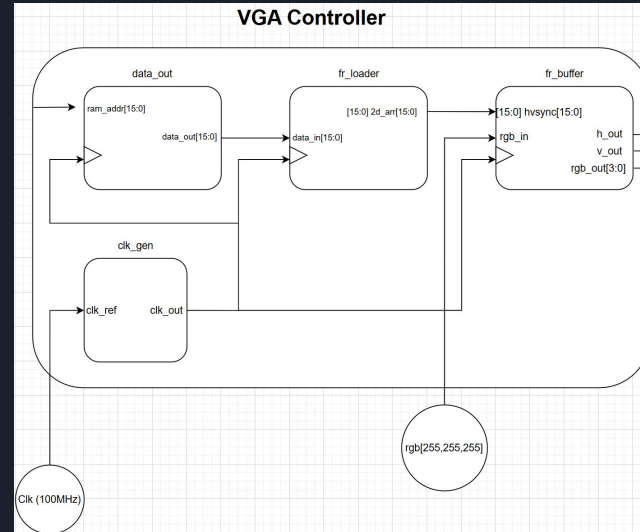
So... What's Changed?

- Updated Block diagram to show the PMOD uses
- Updated count on switches



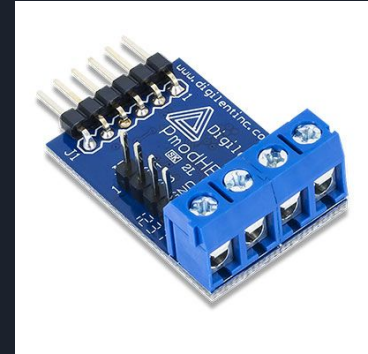
So... What's Changed?

The team has updated the VGA Controller Block Diagram to provide an in-depth view into the mechanics that will be used to read the address from the main FPGA code and output the data contained to the VGA monitor.



So... What's Changed?

- UART Bridge will be built using the PMOD header extension. This PMOD allows the FPGA to be able to have 4 working GPIO's. In the case of this project, we will use 2 of the header inputs as a RX and TX line.
- Although the FPGA already has a built in FPGA bridge, using the header extension allows the project to have a lot more control over the UART transmission.



So... What's Changed?

```
module memory(  
    input clk,  
    input [7:0] m_add, // Address for reading  
    input [7:0] ead,   // Address for writing  
    input [7:0] data_in, // Data to be written  
    input ewr,         // Write enable  
    input mrd,         // Read enable  
    output reg [7:0] temp1 // Output data  
);  
  
    reg [7:0] memr [0:255]; // 256-byte memory array  
  
    always @(posedge clk) begin  
        if (ewr) begin  
            memr[ead] <= data_in; // Write input data to the specified address  
        end  
        if (mrd) begin  
            temp1 <= memr[m_add]; // Read data from the specified address  
        end  
    end  
endmodule
```

The team has developed a basic memory read/write function once the data has been formatted from UART.

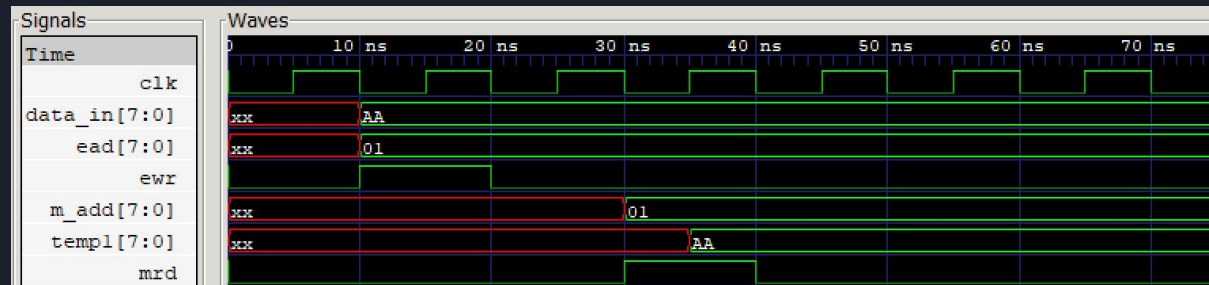
This will continue to be updated until it can store the quantity of data produced from the OBD-II module.

The data stored will continue to be format in a readable form for VGA transmission and output as the team works through the design process.

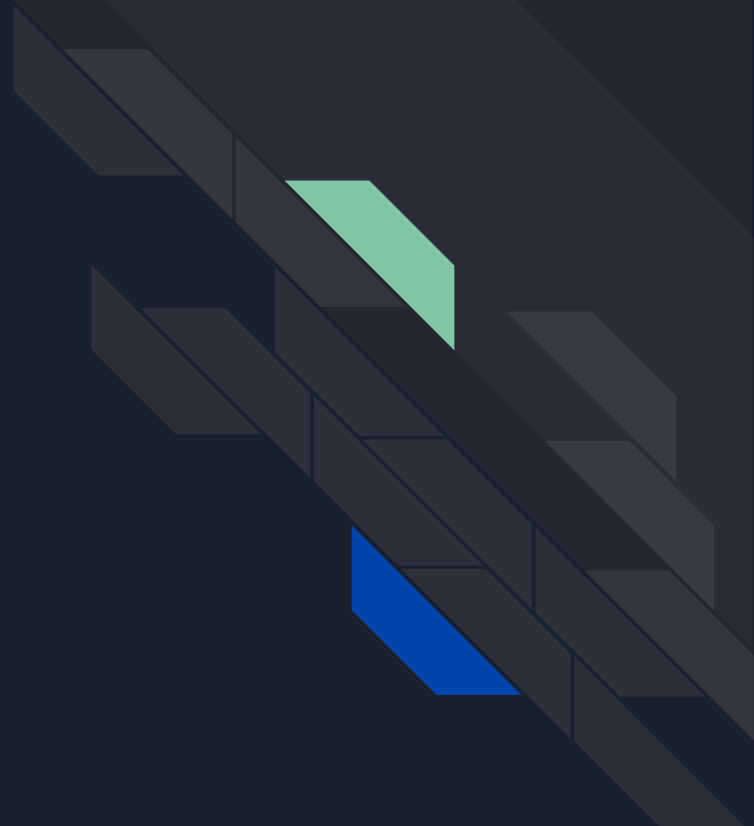
So... What's Changed?

Below is the waveform diagram of the aforementioned code through a self-designed testbench.

- Data_in: Data to be written when write is enabled (ewr)
- Ead: Address for writing
- Temp1: Output data when read function is enabled (mrd)
- M_add: Address for reading



Timeline

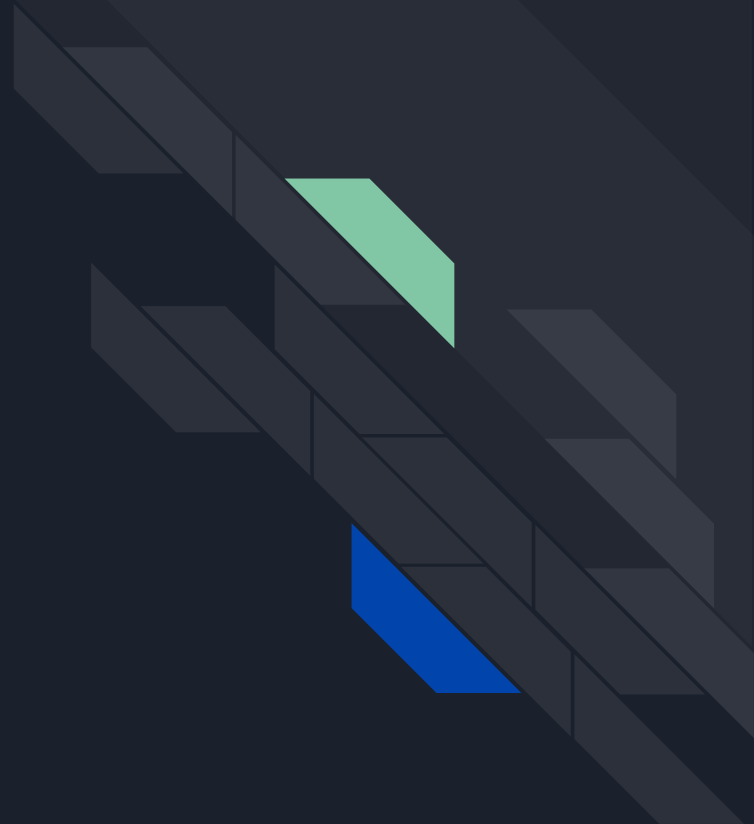




Timeline

	Trent (FPGA Main Code, VGA Controller)	Abhi (OBD-II Bridge, UART Controller)
Design Presentation	Research read/write in verilog, foundation for OBD to VGA com	Finish UART controller - Read Module + FIFO - Write Module + FIFO Buy Hardware
Preliminary Report	Finish storage component, shift to outputting to VGA	Be able to communicate with car via FPGA, Store ECU output in RAM FIFO
Final Report/Video	Complete OBD to VGA output and format UI and display, debug issues	Fully Integrate VGA and FPGA. Complete Project

Design Efficacy

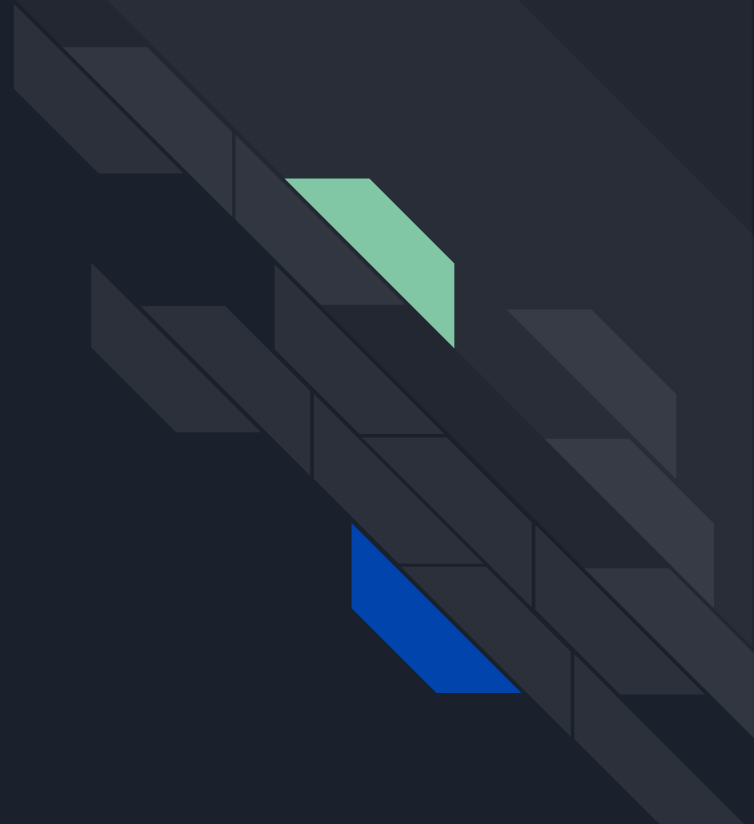




Design Efficacy

- Proper efficient UART data transfers
- FPGA Resource Optimization
- Scalability of project
- Robust/Easy to understand Display onto Monitor

Debug Strategy





Debug Strategy

- Testbench Approach For General Modules/Communications
 - Take every module and test in blocks (UART controller would technically have about 7 or 8 testbenches)
 - Start small and build up
 - Test in iVerilog as much as possible for faster development
 - Only move to Vivado when moving to hardware synthesis and implementation
 - Testbenches are self-checking for quick analysis
- Testbench Approach for Memory
 - All of the above
 - Testing memory overflow/underflow
 - Using the ILA in Vivado to check correct placement of memory

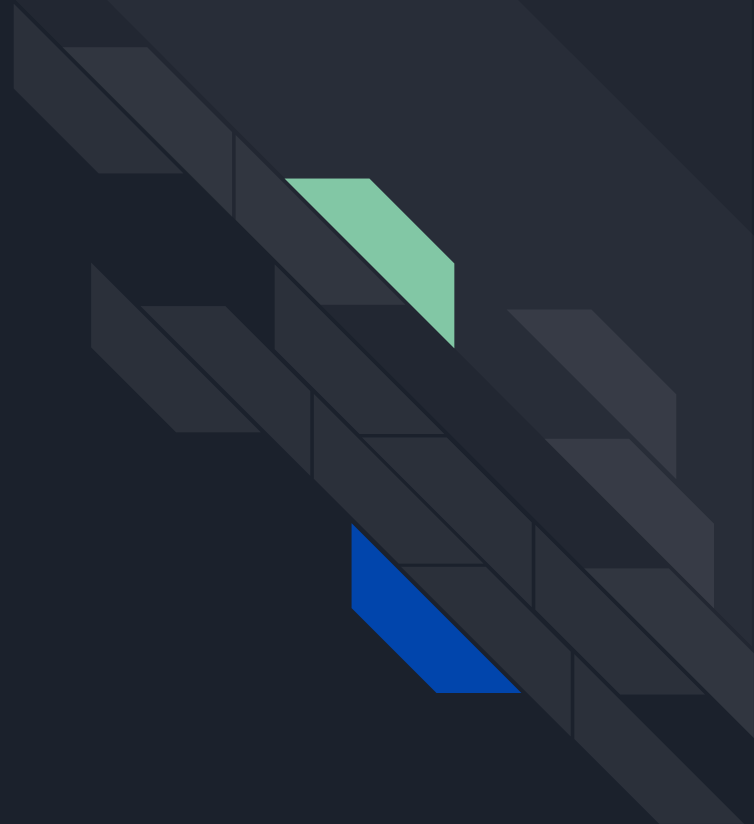


Debug Strategy

We have come up with a relative process to debug/test our project throughout development:

- Divide and conquer approach to design
 - Allows for independent development and testing
- Each designated member will complete a method or program (depending on length and difficulty)
 - Once each defined section is completed, a testbench file will be constructed to determine any faults or missteps within said program.
- Allows for construction and debugging of code without a level of interdependence that would be detrimental to the completion of the project

Project Checkoff



Power Design	This covers how we power the UART module as well as the FPGA within this design	FPGA is powered by laptop and UART module is powered off additional DC supply	FPGA is powered by laptop and UART module finds internal power source, no external power source for UART mod	FPGA is powered either by car ECU or external battery, UART module is powered by OBD-II
UI Design	This will cover how the user sees and digests information via the monitor	Monitor displays pure rev data, non-formmatted	Monitor displays formmatted data such as average temp and rpm data, as well as either a graph of rpm data or temp data	Monitor displays formmatted data such as average temp and rpm data, as well as a graph of rpm data and temp data. All data should be shown with explanations to user
Latency Processing	Since the serial communcation is UART, there will always be some sort of latency, but this project can attempt to do some latency design	Project reads information with ECU and FPGA latency	Project reads information with only ECU latency, FPGA works fast to avoid any latency on its end	Project reads information with only ECU latency, FPGA works fast to avoid any latency on its end
ECU Interaction	This will cover how the project will communciate with the cars computer	FPGA can only read information, errors in transmission are expected	FPGA can only read information, no errors in transmission	FPGA not only can read information but can clear and write information a well
Product Design	This will be how the FPGA and controllers are stored	All wires are connected but loosely stored	Wires and FPGA board are stashed in box so product feels complete	Wires, FPGA, and UART module are stored away in 3D Printed box with outlets for DB9 cable

Questions?

